

Conoscere R

1.1 Cosa è R e perché proprio R?

Dagli anni settanta (del secolo scorso) e fino ai nostri giorni sono state sviluppate tecniche statistiche che sempre più necessitano di supporto informatico sia per la mole di calcoli richiesti, che per l'ampiezza dei dataset e, non ultima, per la necessità di visualizzare informazioni. La statistica applicata si snoda lungo due grandi filoni: da un lato quello della stima, ovvero il problema di ricostruire attraverso i dati i parametri caratterizzanti un modello, dall'altro quello di riconoscere e possibilmente interpretare, strutture intrinseche ai dati, ovvero si cerca di far parlare i dati senza avere per le mani un modello già disponibile. Le tecniche di statistica computazionale e soprattutto l'analisi grafica suggeriscono spesso delle risposte importanti a questi problemi. Ecco quindi che alla fine del secolo scorso i Bell Laboratories (prima AT&T oggi Lucent Technologies) hanno deciso di sviluppare un nuovo ambiente per l'analisi statistica che permettesse, oltre a quanto già citato, anche di sperimentare nuovi modelli e idee statistiche. Nasce quindi il linguaggio S che venne poi concesso in esclusiva alla Insigntful Corporation che ne ha fatto un software di successo noto con il nome di S-PLUS.

Solo di recente, grazie anche all'avvento della Rete, è nato il progetto Open Source di R. I due pacchetti S-PLUS ed R non sono cloni ma più propriamente cugini in quanto affondano le proprie radici nel progetto originale di S di cui sono entrambi *dialetti*. Dire che sono cugini vuol dire che nella stragrande maggioranza dei casi, l'utente medio non scorge evidenti differenze nell'uso delle funzioni e del relativo output, mentre ve ne sono ad un livello meno superficiale.

Nato quindi per offrire, a costo zero, un ambiente statistico di prima qualità R¹ è stato sempre sviluppato in modo tale da mantenere la massima compatibilità con S-PLUS. I due software non si contrappongono da antagonisti ma anzi si supportano in modo naturale l'un l'altro. Dove le risorse economiche lo consentono S-PLUS è ancora il software d'utilizzo primario ma sempre più spesso i

¹Mentre è quasi certo che il nome attribuito al linguaggio S derivi in qualche modo dal più noto linguaggio C ('see' in inglese) c'è tuttora il forte sospetto che il nome R derivi dalle iniziali del nome dei due sviluppatori originali (Robert Gentleman e Ross Ihaka). I due, benché più volte interpellati sull'argomento forniscono inamabilmente risposte evasive alla domanda.

laboratori delle università preferiscono installare licenze gratuite di R per poter svolgere attività (non solo) didattica di alta qualità mettendo, al contempo, gli studenti in grado di lavorare da casa propria con una copia personale, non di frodo, del software. Quindi da un lato R veicola utenti professionali verso l'utilizzo di S-PLUS e al contempo il secondo stimola lo sviluppo del primo e avvicina molti statistici alle più moderne tecniche di analisi dei dati a costo zero. A questo si aggiunge, che uno dei massimi sforzi dell'*R Development Core Team*, cioè il gruppo di sviluppatori di R, è quello di far sì che R funzioni sul maggior numero di sistemi operativi e architetture hardware mantenendo la massima compatibilità tra le varie piattaforme.

Questo volume quindi, contrariamente a molti altri testi di riferimento che descrivono il linguaggio S (più precisamente parlano di S-PLUS) citando R come possibile estensione dell'audience, è interamente basato su R poiché è rivolto in primo luogo agli studenti delle classi di laboratorio di Statistica. È ovvio, per quanto detto sopra, che può essere utilizzato anche da utenti S-PLUS, ma non è questo l'obiettivo principale del volume.

1.1.1 Le caratteristiche peculiari di R

R è un ambiente integrato che permette di elaborare dati, eseguire calcoli ed effettuare rappresentazioni grafiche. Le sue caratteristiche principali possono essere elencate di seguito

- si compone di un insieme di strumenti per l'analisi statistica dei dati;
- si tratta di un linguaggio pensato per descrivere modelli statistici anche estremamente complessi;
- permette la rappresentazione grafica (a video ma anche su supporti tipici dell'editoria come file Postscript, PDF ecc.) di dati;
- è un linguaggio *object-oriented* (come C++ o Java) che può essere facilmente esteso dall'utente finale (e questo avviene sempre più frequentemente);
- (non ultimo) è interamente gratuito (sotto la licenza GNU General Public Licence della Free Software Foundation) e *Open Source*, ovvero ciascuno può avere accesso al codice interno di R ed eventualmente proporre modifiche.

Si parla quindi di *ambiente* e non di software perché tutto è costruito attorno al linguaggio di programmazione R a partire dagli *oggetti*. Come vedremo più avanti, ogni oggetto (vettore, dataset, tabella, grafico, un modello lineare ecc.) viene trattato dalle funzioni di R con uno specifico *metodo* e nuovi metodi possono essere implementati per ampliare, quasi senza limiti, le possibilità delle stesse funzioni. Tradizionalmente invece i software sono costruiti attorno a routines molto strutturate e spesso troppo specializzate che possono risultare in alcuni contesti non sufficientemente flessibili.

1.1.2 Come interagire con R

R può essere utilizzato tramite linea di comando e tipicamente questo avviene in

ambiente UNIX, o attraverso un'interfaccia grafica. Quest'ultima differisce sensibilmente da un'implementazione all'altra (MS-Windows, MacOS, X11) ed in genere è solo di supporto ad operazioni ricorrenti come leggere o scrivere un file e cose del genere. Il motivo principale per cui non vi è uno standard, se non alcune linee guida essenziali, è che R prevede un utilizzo interamente via terminale. Non ci sono quindi prefissati schemi (ovvero finestre² piene di bottoni e gadgets vari) per interloquire con le funzioni di R ma si deve ricorrere sempre alla linea di comando. Tutto quanto diremo si applica a qualsiasi implementazione di R poiché ci concentreremo interamente su quella che viene chiamata "R Console", ovvero la finestra (o la shell nel caso di sistemi UNIX-like) in cui l'utente interagisce con R. Supporremo da qui in avanti che R sia stato correttamente installato e se avete utilizzato il CD-ROM³ allegato questa operazione non dovrebbe aver creato problemi.

All'avvio di R, nella Console appare qualcosa di simile a quanto segue, con l'unica differenza relativa alla versione e alla data di rilascio del software.

```
R : Copyright 2002, The R Development Core Team
Version 1.6.1 (2002-11-01)
```

```
R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.
```

```
R is a collaborative project with many contributors.
Type 'contributors()' for more information.
```

```
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for a HTML browser interface to help.
Type 'q()' to quit R.
```

>

Tutti i comandi devono essere inseriti dopo il prompt ">". All'avvio R ci ricorda come fare ad uscire: il comando "q". In genere esiste anche l'analogo comando attraverso i menu dell'interfaccia grafica. Quando lo utilizzeremo R ci chiederà se vogliamo salvare un'immagine del *workspace*⁴. Il workspace contiene tutti i risultati delle elaborazioni, i dati, le variabili e molte altre informazioni che potrebbero venirci utili la prossima volta che intendiamo utilizzare R. Se salviamo il workspace la prossima volta che utilizziamo R, questo verrà caricato in modo automatico e si potrà proseguire dal punto in cui ci eravamo fermati con le analisi. Si tenga presente che, se non diversamente specificato, R usa la directory di lavoro corrente per salvare i dati in modo automatico. Ogni utente può invece

²L'interfaccia grafica delle funzioni viene, da alcuni, considerata al contrario il punto di forza di S-PLUS.

³Il CD-ROM doppio formato MS-Windows e Macintosh permette di installare R per questi due sistemi operativi. Si veda l'appendice B per i dettagli.

scegliere di spostarsi da una directory all'altra tramite il comando `setwd`, salvare il proprio workspace nella directory prescelta e ricaricarlo nella sessione successiva. Mostriamo a breve, quando avremo materiale per farlo, un esempio di quanto qui accennato.

Un comando con cui è bene familiarizzare è `help` (comando) o, equivalentemente, `?comando`. Per esempio, i comandi `?q` e `help(q)` forniscono lo stesso risultato, cioè chiedono ad R di aprire una nuova finestra contenente una guida sul comando di cui si stanno richiedendo informazioni. Questa finestra contiene una descrizione completa del comando e, più importante, in fondo vengono riportati, quando questo ha senso, una serie di esempi. È chiaro che inizialmente non sappiamo di cosa chiedere l'help ma già il comando `help` ci aiuta ad andare avanti. Per alcuni elementi del linguaggio base di R ottenere l'help è leggermente più complicato. Per esempio, se si vuole ottenere l'help dell'istruzione `for`, la funzione che, vedremo, si occupa di iterare un ciclo, è necessario aggiungere le virgolette, ovvero scrivere

```
> ?"for"
```

se avessimo scritto semplicemente `?for` avremmo ottenuto quanto segue

```
> ?for
+
```

R non riesce ad interpretare il comando, ed infatti ci risponde con un "+". Il simbolo + viene usato da R per segnalare che l'input inserito non è completo e questo accade se la struttura dei nostri comandi è molto lunga. Quando per errore ci si trova in questa situazione si può uscire dall'*empasse* digitando "." o "").

Gli esempi che si trovano alla fine di una finestra di help possono essere eseguiti tramite il comando `example`. Infatti

```
> example(plot)
```

mostra gli esempi del comando `plot` nel device grafico di default per R che verrà aperto per l'occasione⁴. Un comando che è bene esplorare subito è `demo`. Questo comando si occupa di mostrare alcune caratteristiche delle principali funzioni di R. Scrivendo nella Console

```
> demo()
```

in realtà non viene mostrato nulla se non una finestra in cui compare un elenco di possibili opzioni. Per esempio, `demo(graphics)` mostra una rassegna delle capacità grafiche di R. Vi consigliamo di provare!

⁴In un sistema UNIX senza interfaccia grafica il device corrente è `postscript` e quindi verrà creato un file dal nome `Rplots.ps` nella directory di lavoro. Sotto i sistemi Windows e MacOS il device grafico di default è una finestra.

1.2 I fondamentali di R

1.2.1 Vettori e variabili

R lavora in generale con dati strutturati. La più semplice struttura disponibile sono gli scalari e i vettori. Il primo passo consiste nell'assegnare (comando `<-`) ad una variabile un certo valore. Il comando

```
> x <- 4
```

assegna alla variabile `x` il valore 4. Per sapere cosa contiene `x` basterà scrivere

```
> x
[1] 4
```

R ci risponderà fornendo il valore assunto da `x` premettendo all'output `[1]`. Ciò vuol dire che R interpreta anche gli scalari come vettori di lunghezza 1. Quindi nel precedente output 4 risulta essere il primo elemento del vettore `x`. Se vogliamo creare un vettore⁵ `y` contenente i numeri 2, 7, 4 e 1 il comando da dare ad R è il seguente

```
> y <- c(2, 7, 4, 1)
```

Se chiediamo ad R chi è `y`

```
> y
```

R risponderà stampando i valori

```
[1] 2 4 7 1
```

Si noti che ancora una volta R precede l'output con `[1]`, semplicemente per indicare a cosa corrisponde il primo elemento del vettore `y`. È chiaro quindi che 4, 7 e 1 sono, rispettivamente, il secondo, il terzo e il quarto elemento di `y`.

Il comando `"c"` serve per concatenare gli elementi che vengono forniti come argomento. Questa è la procedura che abbiamo utilizzato per creare un vettore di elementi 2, 4, 7 ed 1. Successivamente lo abbiamo assegnato alla variabile `y`. Notare che l'assegnazione avviene tramite una freccia e non, come usualmente si fa con altri linguaggi di programmazione, attraverso il simbolo `"="`. Benché il simbolo `"="` sia consentito in R questo è oggi fortemente sconsigliato. Tale freccia è necessaria perché specifica quale quantità viene assegnata ad un'altra. I comandi

```
> y <- c(2, 7, 4, 1)
```

```
e
```

⁵Attenzione! con il termine "vettore" in R non ci riferiamo alla nozione di vettore usuale dell'algebra lineare ma, semplicemente, ad una stringa di valori consecutivi benché R rappresenti queste stringhe per riga nella Console.

> c(2,7,4,1) -> y
sono equivalenti. Un altro comando che useremo spesso è `ls` che fornisce l'elenco di tutti gli oggetti presenti nel workspace di R.

> ls()
[1] "x" "y"

Si possono eseguire assegnazioni anche tramite il comando

> assign("y", c(2,7,4,1))

ma l'operazione risulta solo più macchinosa ed ha senso solo quando gli oggetti coinvolti sono per loro natura complessi o, meglio, quando i nomi delle variabili sono essi stessi delle variabili. Il codice che segue crea nove nuove variabili con i nomi "V1", "V2", ..., "V9".

```
> for(i in 1:9) assign(paste("V", i, sep=""), i)
> ls()
[1] "V1" "V2" "V3" "V4" "V5" "V6" "V7"
[8] "V8" "V9" "1" "x" "y"
```

Non ci soffermiamo sul ciclo `for` di cui parleremo a breve mentre notiamo che nella linea di codice `R` appena vista, è stata impiegata la funzione `paste`. Questa funzione di occupa di concatenare due (o più) stringhe in una sola. Se gli oggetti non sono stringhe, come nel nostro caso, `R` procede prima a trasformarli in stringhe. L'opzione `sep` serve a specificare che tipo di separatore si vuole inserire tra gli oggetti. Le linee di codice che seguono dovrebbero chiarire quanto detto.

```
> paste("A", "B", 2, "C", sep="*")
[1] "A*B*2*C"
> paste("A", "B", 2, "C", sep=" ")
[1] "AB2C"
> paste("A", "B", 2, "C", sep=" * ")
[1] "A * B * 2 * C"
```

1.2.2 Matrici e operazioni algebriche

R esegue operazioni sia vettoriali che scalari. Siano x ed y come nella precedente sezione. Se diamo il comando

```
> x*y
```

il risultato sarà

```
[1] 8 28 16 4
```

il che non stupisce affatto coincidendo con la usuale nozione di prodotto di uno scalare per un vettore. Se invece scriviamo

```
> y*y
```

come risultato otteniamo

```
[1] 4 49 16 1
```

cioè il prodotto termine a termine degli elementi di y con se stessi, ovvero il quadrato degli elementi di y . Se vogliamo utilizzare l'aritmetica dell'algebra lineare si devono utilizzare gli operatori opportuni. In tal caso R interpreta i suoi vettori come vettori colonna. Per esempio, i prodotti del tipo $y' \times y$, con y vettore colonna, si scrivono:

```
> t(y) %*% y
[1] 30
[1] 4 49 16 1
```

mentre se vogliamo ottenere $y \times y'$ si dovrà dare il comando

```
> z <- y %*% t(y)
```

e la matrice risultante sarà

```
> z
[1,] 4 14 8 2
[2,] 14 49 28 7
[3,] 8 28 16 4
[4,] 2 7 4 1
```

Per convenzione l'operazione

```
> y %*% y
```

è equivalente a

```
> t(y) %*% y
```

Il comando `"t"` effettua la trasposizione di un vettore/matrice. Per creare una matrice ex-novo si utilizza il comando `matrix`

```
> a <- matrix(1:30, 5, 6)
```

che crea una matrice di dimensione 5×6 riempiendola dei numeri da 1 a 30. Il comando `matrix` permette di creare una matrice in cui il primo argomento sono l'insieme di valori che costituiranno la matrice e il secondo e terzo parametro sono il numero di righe e colonne rispettivamente. Nell'esempio precedente `-1:30` costituisce un vettore rappresentato dalla sequenza di numeri interi da -1 a 30. Infatti, se scriviamo

```
> -1:30
[1] -1 0 1 2 3 4 5 6 7 8 9 10
[13] 11 12 13 14 15 16 17 18 19 20 21 22
[25] 23 24 25 26 27 28 29 30
```

è più evidente il modo in cui R ci presenta il contenuto del vettore. Infatti, nella posizione [1,3] c'è il numero 11 e nella [2,5] il numero 23. Si tenga presente che la convenzione in R è quella di far partire gli indici dei vettori con 1. Questo può creare confusione a chi è abituato a lavorare in C dove la convenzione è quella di far partire gli indici da 0. Si tenga presente però che la convenzione di R è coerente con la notazione usuale dell'algebra.

Tornando al comando `matrix`, si deve tener presente che R riempie le matrici per colonna. Se si vuole forzare il riempimento per riga si deve specificare l'opzione `byrow = TRUE`. Vediamo due esempi a riguardo di quanto detto

```
> # Riempimento per colonna
> a <- matrix(1:30, 5, 6)
> a
     [,1] [,2] [,3] [,4] [,5] [,6]
[1,]    1    6   11   16   21   26
[2,]    2    7   12   17   22   27
[3,]    3    8   13   18   23   28
[4,]    4    9   14   19   24   29
[5,]    5   10   15   20   25   30

> # Riempimento per riga
> matrix(1:30, 5, 6, byrow=TRUE)
     [,1] [,2] [,3] [,4] [,5] [,6]
[1,]    1    2    3    4    5    6
[2,]    7    8    9   10   11   12
[3,]   13   14   15   16   17   18
[4,]   19   20   21   22   23   24
[5,]   25   26   27   28   29   30
```

Nelle linee di codice precedente abbiamo impiegato il simbolo # che in R rappresenta un indicatore di commento. Si possono inserire questi commenti per spiegare al lettore delle nostre linee di codice (l'equivalente in BASIC è REM, in FORTRAN c ecc.). Quando R incontra il simbolo # ignora tutto quello che segue. Il comando `c` può essere applicato anche alle matrici ovviamente

```
> t(a)
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    2    3    4    5
[2,]    6    7    8    9   10
[3,]   11   12   13   14   15
[4,]   16   17   18   19   20
[5,]   21   22   23   24   25
[6,]   26   27   28   29   30
```

Se si vuole costruire una matrice piena di zeri o di uno si può egualmente utilizzare il comando `matrix(0,2,3)` o `matrix(1,2,3)` rispettivamente. Se si vuole creare una matrice senza assegnare valori precisi agli elementi si può scrivere `matrix(,2,3)`

```
> matrix(,2,3)
     [,1] [,2] [,3]
[1,] NA  NA  NA
[2,] NA  NA  NA
```

Come si vede abbiamo introdotto una novità: omettendo il primo argomento R ha riempito la matrice di valori NA. Il simbolo NA è utilizzato da R per definire i valori mancanti (*missing*) spesso presenti in molti dataset. Dove NA è l'acronimo di *Not Available*. Per inciso si noti che la gran parte delle funzioni di R prevede la possibilità di trattare separatamente o in modo automatico i dati mancanti.

Torniamo all'algebra lineare. Sembra poco intuitivo dover utilizzare l'operatore `%*%` al posto del semplice prodotto `*` per calcolare il prodotto vettoriale. In realtà tale convenzione è pensata per velocizzare alcuni tipi di calcoli e per rendere uniforme la notazione. In altri linguaggi, ad esempio Matlab, il comando

```
a^2
```

esegue la potenza seconda della matrice a, mentre `log(a)` esegue il calcolo del logaritmo termine a termine sugli elementi di a. In R sia la potenza che il logaritmo appartengono alla stessa classe di funzioni e vengono applicate in modo coerente termine a termine alle matrici. Comunque, se si vuole lavorare correttamente con l'algebra lineare si devono creare i vettori come se fossero delle matrici in cui una delle due dimensioni è pari ad 1. Per esempio i comandi

```
> x <- matrix(c(1,2,3,4),1,4)
> x
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
```

e

```
> y <- matrix(c(1,2,3,4),4,1)
> y
     [,1]
[1,]    1
[2,]    2
[3,]    3
[4,]    4
```

creano un vettore riga *x* ed un vettore colonna *y* rispettivamente di dimensione 1×4 e 4×1 . Il prodotto

```
> x*y
```

darà come risultato

```
Error in x * y : non-conformable arrays
```

in quanto ora gli array, cioè i vettori, sono quelli dell'algebra lineare e non è possibile usare gli operatori che eseguono operazioni termine a termine, mentre

```
> x %*% y
```

e

```
> y %*% x
```

forniranno i risultati corretti, rispettivamente uno scalare e una matrice. Si ricordi che i simboli $+$ $-$ $/$ $*$ agiscono elemento per elemento sui vettori intesi come stringhe. Supponiamo di disporre dei vettori $x = (1, 2, 3, 4)$ e $y = (2, 4, 6, 8)$ e di voler generare quattro nuovi vettori v_1 , v_2 , v_3 e v_4 i cui elementi sono rispettivamente la somma, la differenza, il prodotto ed il rapporto degli elementi dei due vettori x ed y . Si può procedere come segue:

```
> x <- c(1, 2, 3, 4)
> y <- c(2, 4, 6, 8)
> v1 <- x+y
> v2 <- x-y
> v3 <- x*y
> v4 <- x/y
> v1
[1] 3 6 9 12
> v2
[1] -1 -2 -3 -4
> v3
[1] 2 8 18 32
> v4
[1] 0.5 0.5 0.5 0.5
```

Esistono inoltre altri operatori che permettono di calcolare le potenze, eseguire divisioni che restituiscono solo interi e il calcolo del modulo (da non confondere con il valore assoluto). Per i dettagli ed esempi si utilizzi

```
> ? "+^"
> example("+^")
```

1.2.3 Accedere agli elementi dei vettori

Poiché serviranno diffusamente nel seguito vediamo come è possibile costruire vettori di indici. Lo scopo di creare un vettore i cui elementi sono degli indici, sarà quello di estrarre o manipolare gli elementi di un secondo vettore. Supponiamo di voler creare un vettore x che contenga i primi $n = 4$ numeri naturali. L'istruzione da dare a R è la seguente

```
> x <- 1:4
> x
[1] 1 2 3 4
```

analogamente $x <- -3:8$ produrrà

```
[1] -3 -2 -1 0 1 2 3 4 5 6 7 8
```

Il comando `1:8` è equivalente al comando `seq(1, 8)` solo che questa seconda versione offre in più la possibilità di specificare il passo della successione (si noti che `seq` è l'abbreviazione di *sequence* = successione). Il passo può anche essere un numero non intero

```
> seq(-3, 6, 2)
[1] -3 -1 1 3 5
> seq(-3, -1, .33)
[1] -3.00 -2.67 -2.34 -2.01 -1.68 -1.35 -1.02
```

Creiamo dunque un vettore *sequenza* e costruiamone una matrice

```
> sequenza <- -3:8
> sequenza
[1] -3 -2 -1 0 1 2 3 4 5 6 7 8
> A <- matrix(sequenza, 2, 6)
> A
     [,1] [,2] [,3] [,4] [,5] [,6]
[1,]  -3  -2  -1  0  1  2
[2,]  3  4  5  6  7  8
```

Se si vuole accedere al terzo elemento della sequenza basterà scrivere

```
> sequenza[3]
[1] -1
```

Se invece vogliamo il termine $a_{2,3}$ della matrice A basterà scrivere

```
> A[2,3]
[1] 2
```

Se si vuole ricavare un'intera riga o colonna della matrice A basterà scrivere

```
> # seconda riga della matrice A
> A[2,]
[1] -2 0 2 4 6 8
> # sesta colonna della matrice A
> A[,6]
[1] 7 8
```

Un altro comando che può tornare utile per cercare elementi all'interno di un vettore è `which`. Per esempio, se vogliamo sapere quali sono gli indici corrispondenti agli elementi del vettore x

```
> x <- -3:8
> x
[1] -3 -2 -1 0 1 2 3 4 5 6 7 8
```

minori di 2, si utilizza il comando `which` nel seguente modo

```
> which(x<2)
[1] 1 2 3 4 5
```

oppure i quali sono compresi tra -1 incluso e 5 escluso useremo `which` come segue

```
> which((x >= -1) & (x < 5))
[1] 3 4 5 6 7 8
```

dove "x" è l'operatore logico *and*. Si può utilizzare anche l'operatore "!" (or logico) per ottenere, ad esempio, gli indici degli elementi di x più grandi di 1 o più piccoli di -2

```
> which((x < -2) | (x > 1))
[1] 1 6 7 8 9 10 11 12
```

e per ottenere gli elementi di x che interessano basta scrivere quanto segue

```
> x[z]
[1] -3 2 3 4 5 6 7 8
```

1.2.4 Il workspace

Arrivati a questo punto dovremmo aver creato diversi oggetti. Utilizzando il comando `ls` dovremmo vederli tutti nel workspace.

```
> ls()
> ls()
[1] "A"      "V1"      "V2"
[4] "V3"      "V4"      "V5"
[7] "V6"      "V7"      "V8"
[10] "V9"      "a"       "i"
[13] "sequenza" "V1"      "V2"
[16] "V3"      "V4"      "x"
[19] "y"      "z"
```

Si noti che compaiono sia a che A. Questo avviene perché R è un linguaggio in cui le lettere minuscole e maiuscole sono considerate come differenti. Ciò è comune ad altri linguaggi di programmazione come C, Java ecc. mentre non lo è in molti altri software statistici.

Possiamo salvare il contenuto del workspace con il comando `save.image()`. Questo comando permette di memorizzare su file l'intero contenuto del workspace. Il nome del file è per convenzione `.RData` e viene memorizzato nella directory corrente di lavoro. I comandi `getwd` e `setwd` permettono, rispettivamente, di conoscere la directory di lavoro corrente e di cambiarla. Il modo in cui si specificano i percorsi delle directory dipende dalla versione del sistema operativo su cui gira R, si consiglia quindi di leggere il file di help delle due funzioni. Si può decidere di salvare (anziché tutto) solo qualcuno degli elementi. Per far questo si può utilizzare il comando `save` specificando la lista degli oggetti da salvare e il file in cui preferiamo salvarli. Per esempio,

```
> save(x, A, file="prova.rda")
```

memorizza nel file `prova.rda` gli elementi `x` e `A` del workspace. Gli elementi del workspace possono essere cancellati con il comando `rm`

```
> rm(v1, v2, v3, v4)
> ls()
[1] "A"      "V1"      "V2"
[4] "V3"      "V4"      "V5"
[7] "V6"      "V7"      "V8"
[10] "V9"      "a"       "i"
[13] "sequenza" "x"      "y"
[16] "z"
```

oppure tutti in blocco

```
> rm(list=ls())
> ls()
character(0)
```

Ora che il workspace è vuoto possiamo ricaricare quanto abbiamo salvato all'interno del file `prova.rda`. Il comando da utilizzare è `load`

```
> load("prova.rda")
> ls()
[1] "A" "x"
> A
      [,1] [,2] [,3] [,4] [,5] [,6]
[1,]    -3    -1     1     3     5     7
[2,]     -2     0     2     4     6     8
> X
Error: Object "X" not found
> x
[1] 1 2 3 4
```

Nelle linee di codice qui sopra sbadatamente abbiamo chiesto a R di mostrarci il contenuto di `x` anziché quello di `X`. Per i motivi sopra detti `x` e `X` sono oggetti differenti e questo genera un messaggio di errore da parte di R.

1.2.5 Gli oggetti e le tipologie di dati

R dispone di una varietà di tipi di dati sufficientemente diversificati da risultare adeguati nel maggior numero delle analisi. Esistono poi delle tipologie di dati, derivate dall'insieme di base, che possono essere create *ad hoc* dagli utenti. Non ci occuperemo di questi in quanto sono specifici di alcuni pacchetti non trattati nel seguito. La prima cosa da tenere a mente è che ogni cosa in R è un oggetto ed ogni oggetto appartiene ad una classe. Nello specificare i nomi degli oggetti si deve tener presente che ogni nome può essere una sequenza di lettere e numeri purché non inizi con un numero. Per esempio `x0` è un nome ammissibile mentre non lo è `0x`. Inoltre, alcuni nomi sono riservati e sono

```
FALSE TRUE Inf NA NaN NULL
break else for function if in next repeat while
```

Inoltre, i nomi delle funzioni non possono contenere caratteri particolari come `'`, `''` (underscore) mentre possono contenere, come abbiamo già visto, il simbolo `'` (il punto). Ci sono poi alcuni nomi che sarebbe meglio evitare di usare. Per esempio `define` o `c` o un oggetto è un'azione sconsigliata anche se ammessa. Possiamo ora in rassegna vari tipi di oggetti che R può maneggiare:

- `character`: si tratta delle stringhe o sequenze di caratteri;
- `numeric`: sono i numeri reali. Si noti che in R tutti gli oggetti numerici sono in doppia precisione;
- `integer`: numeri interi eventualmente con segno;
- `logical`: sono elementi che assumono solo i valori `TRUE` (vero) o `FALSE` (falso) e `NA` (not available);
- `complex`: i numeri complessi

Tutti questi oggetti possono essere scalari o matrici a due o più vie. Esiste poi il concetto di *lista* che può essere visto come un contenitore di oggetti dei più disparati. Per essere più precisi

- `list`: è un vettore di oggetti di R del tipo visto sopra, anche con diverse tipologie per i diversi elementi.

Costruiamo una lista di oggetti differenti in cui inseriamo un vettore di numeri complessi, un vettore di stringhe, un vettore logico e una matrice.

```
> cmp <- complex(real=1:10, imaginary=-1:9)
> cmp
[1] 1-1i 2+0i 3+1i 4+2i 5+3i 6+4i
[7] 7+5i 8+6i 9+7i 10+8i 1+9i
> str <- c("tizio", "caio", "sempronio")
> str
[1] "tizio" "caio" "sempronio"
> log <- c(TRUE, TRUE, FALSE, FALSE, FALSE)
> log
[1] TRUE TRUE FALSE FALSE FALSE
> A <- matrix(1, 4, 2)
> A
     [,1] [,2]
[1,] 1    1
[2,] 1    1
[3,] 1    1
[4,] 1    1
```

e ora assembliamo tutto in una lista

```
> elle <- list(CPLX = cmp, NOMI = str, BOOL = log,
             matrice = A)
```

e andiamo a scoprire quale mostro abbiamo creato!

```
> elle
$CPLX
[1] 1-1i 2+0i 3+1i 4+2i 5+3i 6+4i
[7] 7+5i 8+6i 9+7i 10+8i 1+9i
$NOMI
[1] "tizio" "caio" "sempronio"
$BOOL
[1] TRUE TRUE FALSE FALSE FALSE
$matrice
     [,1] [,2]
[1,] 1    1
[2,] 1    1
[3,] 1    1
[4,] 1    1
```

Come si nota, nella costruzione di una lista, si possono specificare i nomi degli oggetti. Ora la lista `elle` contiene 4 elementi chiamati rispettivamente `CPLX`, `NOMI`, `BOOL` e `matrice`. Non è obbligatorio specificare tali nomi ma è spesso utile farlo. Se vogliamo accedere al vettore di elementi logici `BOOL` possiamo farlo in due modi: o richiamando l'elemento per nome `c`, in tal caso, si deve utilizzare il simbolo `$` per accedere agli elementi di `elle`

```
> elle$BOOL
[1] TRUE TRUE FALSE FALSE FALSE
oppure, sapendo che si tratta del terzo elemento della lista, possiamo scrivere
> elle[[3]]
[1] TRUE TRUE FALSE FALSE FALSE
si noti l'utilizzo del doppio [ ] per accedere agli elementi della lista. Se invece vogliamo accedere agli elementi degli elementi di una lista, ovvero, se ad esempio, vogliamo ottenere il quarto numero del vettore di numeri complessi CPLX possiamo procedere come segue
> elle$CPLX[4]
[1] 4+2i
> elle[[1]][4]
[1] 4+2i
```

e si osservi come è molto più convoluta la seconda scrittura rispetto alla prima. Dicevamo che ad ogni oggetto appartiene ad una particolare classe. Per ricavare informazioni sulla classe di un oggetto si può utilizzare il comando `str`. Vediamo che tipo di informazioni restituisce il comando `str`

```
> str(cmp)
cplx [1:11] 1-1i 2+0i 3+1i ...
> str(log)
logi [1:5] TRUE TRUE FALSE FALSE FALSE
```

```

> str(str)
chr [1:3] "tizio" "caio" "sempronio"
> str(A)
num [1:4, 1:2] 1 1 1 1 1 1
> str(e1e)
list of 4
$ CPLX : cplx [1:11] 1-1i 2+0i 3+1i ...
$ NOMI : chr [1:3] "tizio" "caio" "sempronio"
$ BOOL : logi [1:5] TRUE TRUE FALSE FALSE
$ matrice: num [1:4, 1:2] 1 1 1 1 1 1

```

Questo comando restituisce informazioni sulla struttura di un oggetto: sia la natura che le dimensioni. In particolare si noti come le matrici sono definite di classe `num`, cioè numerica, se avessimo costruito una matrice di numeri complessi avremmo ottenuto `cplx` per `A`. In quanto fatto sopra abbiamo introdotto un'annotazione: abbiamo definito come `log` un vettore quando invece `log` è la funzione logaritmo di `R` e analogamente per `str`. È meglio evitare di definire degli oggetti con nomi di funzioni che sono già definite in `R` per evitare di incorrere in problemi di questo tipo

```

> log <- 2
> log
[1] 2
> log(2)
[1] 0.6931472
> log <- function(x) x
> log(2)
[1] 2
> log(3)
[1] 3

```

Nell'esempio sopra riportato abbiamo prima definito `log` come uno scalare, poi abbiamo chiamato la funzione `log` (logaritmo) di `R` e infine abbiamo ridefinito quest'ultima come la funzione identità (si legga il paragrafo successivo per i dettagli). Da questo momento in poi, qualsiasi altra funzione o pacchetto di `R` che utilizzi la funzione logaritmo nei suoi calcoli fornirà, chiaramente, risultati inconsistenti.

1.2.6 Le funzioni

Abbiamo più volte utilizzato delle funzioni di `R` senza mai averci guardato realmente dentro. Ogni comando non elementare di `R` è definito come una funzione costruita a partire proprio dai comandi elementari⁶. Costruire una funzione è molto semplice. Costruiamo la funzione `somma` che si occupa di sommare due numeri `a` e `b`. Il codice `R` è il seguente

⁶In realtà anche tutti i comandi elementari sono essi stessi delle funzioni (`<-`, `+` ecc.). Le uniche vere eccezioni sono l'insieme degli oggetti i cui nomi non possono essere ridefiniti e di cui già si è parlato (`break`, `else`, ..., `while`).

```

> somma <- function(a,b) a+b
> somma(2,3)
[1] 5

```

Analizziamo le linee di codice appena scritte: il nome della funzione è `somma` che è stata definita come un oggetto di classe `function`. Questo comando `function` è uno di quelli elementari (si ricordi che è anche uno dei nomi di oggetti proibiti in `R`). Con `function` si specificano gli argomenti racchiusi tra parentesi tonde. Alla fine si definisce lo scopo della funzione la quale, nel nostro caso, si occupa semplicemente di sommare i valori di input. Così come è scritta la funzione non è in grado di verificare se gli argomenti che vengono messi in input siano conformi alla sua natura. Per esempio,

```

> somma("ciro", "piro")
Error in a + b : non-numeric argument to binary operator

```

è quanto ci si deve logicamente aspettare come comportamento da parte di `R`. Le funzioni di `R` in generale sono molto più complesse e al loro interno prevedono il controllo degli argomenti. In particolare modo risulta molto importante per `R`, o meglio per le sue funzioni, sapere di che classe sono gli oggetti passati in input in modo da potersi comportare di conseguenza, cioè applicare un metodo. Per scoprire quali siano gli argomenti di una funzione si può ricorrere all'help della funzione stessa, quando previsto, oppure utilizzare il comando `args` o `str`.

```

> args(somma)
function (a, b)
NULL
> str(somma)
function (a, b)
- attr(*, "source") = chr "function(a,b) a+b"
> args(q)
function (save = "default", status = 0, runlast = TRUE)
NULL

```

Se invece vogliamo capire come è costituito il corpo di una funzione si può utilizzare il comando `body`

```

> body(somma)
a + b

```

Infine, se vogliamo modificare il codice di una funzione già scritta possiamo utilizzare il comando `fix` applicato alla funzione stessa. Si provi ad eseguire `fix(somma)`⁷.

⁷L'implementazione del comando `fix` dipende strettamente dall'interfaccia grafica della piattaforma su cui stiamo utilizzando `R`. Nei sistemi Unix-like gli utenti che lavorano con `R` attraverso Emacs non utilizzano mai la funzione `fix`.

di testo ASCII standard l'insieme dei comandi. Una volta in R sarà sufficiente utilizzare il comando `source`. Supponiamo di creare un file di testo `prova.R` in cui sono contenuti alcune delle funzioni da noi definite, ad esempio la funzione `somma`. Quindi il file conterrà le seguenti linee di testo

```
# la nostra strana funzione somma
somma <- function(a,b){
  if(a>0 | b>0)
    a+b
  else
    -1
}
```

Supponiamo ora di aprire una nuova sessione di R e di voler ricaricare in memoria la funzione `somma` contenuta nel file `prova.R`. Sarà sufficiente scrivere

```
> source("prova.R")
> ls()
[1] "somma"
> somma(2,3)
[1] 5
```

1.2.9 Salvare l'output di R

A volte, anche in relazione all'utilizzo di script esterni, si rende necessario poter memorizzare l'output di una sessione di lavoro con R. Le versioni dotate di interfaccia grafica prevedono questa opzione tramite menu oppure con semplice copia e incolla della finestra R Console. Se però si vuole automatizzare anche questa funzionalità si può utilizzare il comando `sink`. Per esempio, la sequenza di comandi

```
> sink("output.txt")
> cat("\n Hello World!\n")
> sink()
```

apre un file "output.txt" nella directory corrente di lavoro e vi scrive il testo `Hello World!`. Si noti che l'ultimo comando `sink`, senza argomenti, serve a chiudere il file di output.

1.2.10 La directory di lavoro

A seconda delle implementazioni di R utilizzate, la directory può essere differente da quella in cui si trova l'eseguibile di R (questo avviene quasi sempre sotto sistemi Unix). È possibile spostarsi da una directory all'altra attraverso il comando `setwd`, verificare la directory corrente di lavoro attraverso il comando `getwd` e visionarne il contenuto attraverso il comando `dir`. Spesso è conveniente tenere distinte le sessioni di lavoro lavorando su directory differenti e quindi salvare e caricare il contenuto del workspace in queste directory di lavoro.

2

Sintesi dei dati

In questo capitolo tratteremo di quella parte della statistica che si occupa di sintetizzare l'informazione contenuta nei dati grezzi: la statistica descrittiva. In questo contesto lo scopo principale è dare una lettura dei dati in nostro possesso che sia il più immediata possibile senza tentare di estendere i risultati ottenuti ad altre unità statistiche o popolazioni. Di questo genere di estensioni se ne occupa l'inferenza statistica e sarà oggetto dei capitoli successivi. Quindi, indipendentemente dal fatto che si tratti di rilevazioni complete o dati campionari, la nostra analisi si fermerà alla descrizione di questo insieme di dati. Gli strumenti fondamentali che incontreremo sono le distribuzioni di frequenza, le (loro) rappresentazioni grafiche e i principali indici di sintesi che le descrivono.

2.1 Tipologie di dati

In statistica esistono varie tipologie di dati che non corrispondono necessariamente alla natura dei fenomeni statistici cui fanno riferimento. I dati si dividono quindi in due gruppi principali: quelli qualitativi e quelli quantitativi. I **qualitativi** sono così chiamati perché i fenomeni statistici da cui derivano possono essere descritti attraverso attributi (per esempio il "sesso" può essere registrato attraverso i due attributi "maschio" e "femmina") eventualmente posti tra loro in qualche relazione d'ordine (per esempio l'età: "giovane", "adulto", "anziano"). Si parla quindi di fenomeni qualitativi rilevati su **scala nominale** e **ordinale** rispettivamente. In R i dati di questo tipo sono rappresentati come `factor` ed eventualmente, se è presente un ordinamento, si deve specificare anche l'attributo `ordered`.

Supponiamo di avere un gruppo di sette persone, tre uomini e quattro donne, rispettivamente di 15, 16, 45, 55, 75, 15 e 70 anni. Costruiamo i due vettori in R ricodificando le età utilizzando la scala "giovane", "adulto", "anziano".

```
> sesso <- c("U", "U", "U", "D", "D", "D", "D")
> eta <- c("giovane", "giovane", "giovane", "adulto", "adulto",
          "anziano", "giovane", "anziano")
> str(sesso)
chr [1:7] "U" "U" "U" "D" "D" "D" "D"
> str(eta)

```